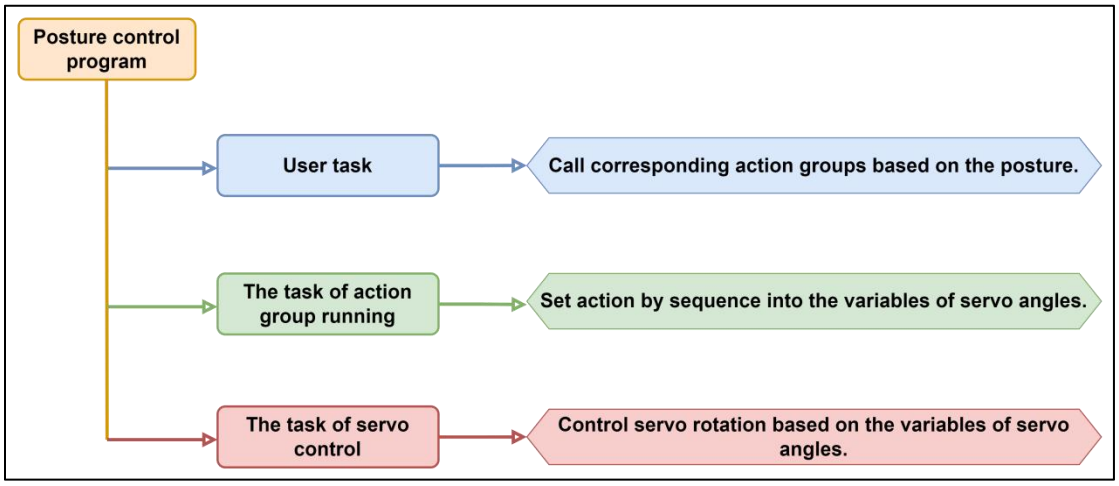
Posture Control

In this section, you will learn how to use the touch sensor to test touch state. And how to use it to realize the action of touching the robotic arm to grasp the square, then to place the square. Meanwhile, control the RGB light to change color.

1. Program Flowchart

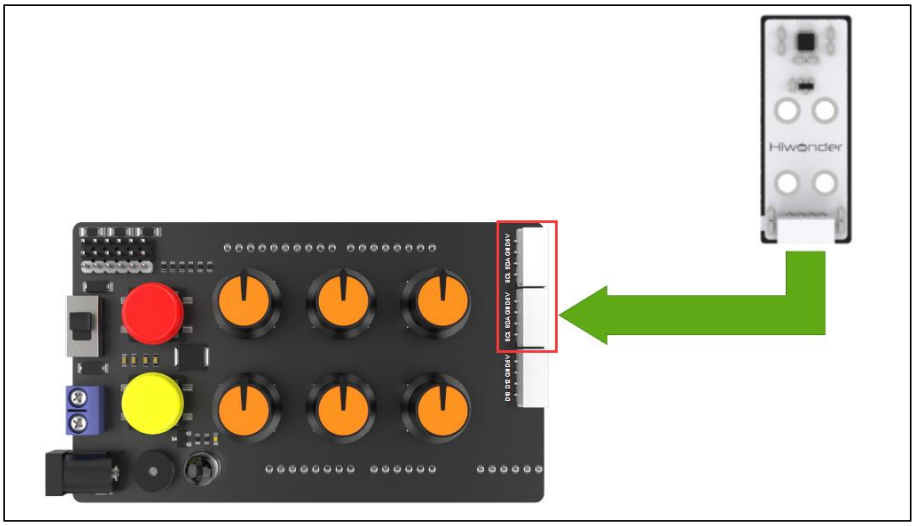


2. Touch Sensor

This sensor primarily utilizes the MPU6050 sensor component. It integrates a 3-axis MEMS gyroscope, a 3-axis MEMS accelerometer, and an expandable Digital Motion Processor (DMP).

2.1 Sensor Wiring

Connect the accelerometer to any I2C interface through 4Pin cable.



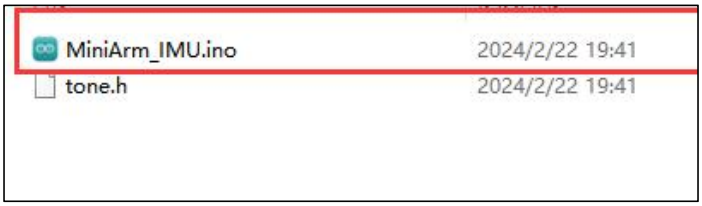
3. Program Download

Note:

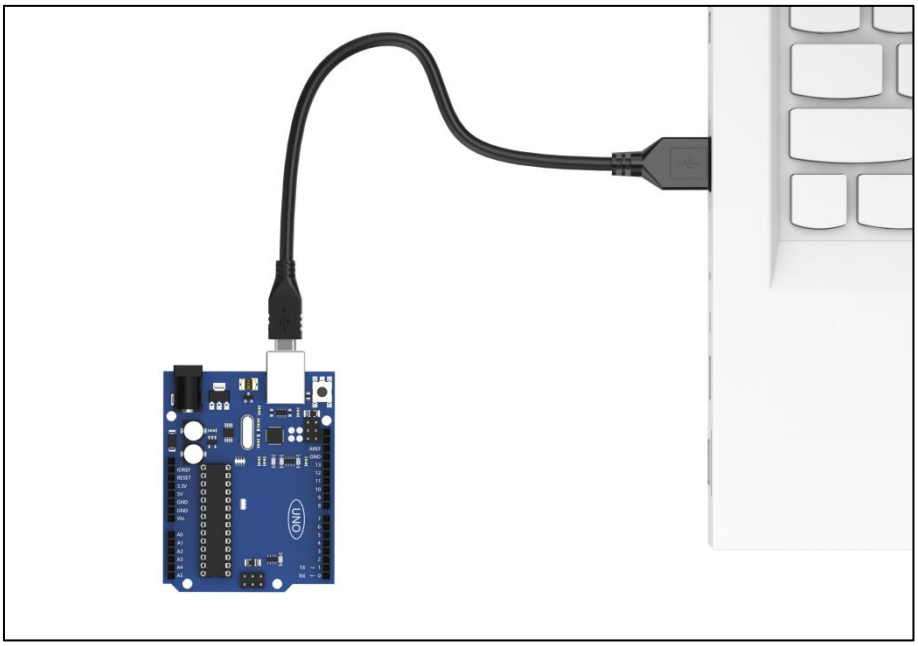
◆ Remove the Bluetooth module before downloading the program, otherwise the serial interface conflict will lead to program download failure.

◆ Please turn the switch of the battery box to “OFF” when connect Type-B download cable to avoid the cable touch the power pins of expansion board by mistake, which may cause a short circuit.

1)Locate “MiniArm_IMU.ino” program file in the same path as document “Program File\MiniArm_IMU”.



2)Connect Arduino to the computer through UNO Type-B cable.



3)Click “Select Development Board” option. The software will automatically test current Arduino serial port, then click to connect.

4)Click to download the program to Arduino, then wait for download to complete.

4. Program Outcome

After powering on, the robotic arm will return to the initial position and then proceed to preparatory action for grasping the block.

Taking the "hiwonder" label on the accelerometer as the positive direction, tilt

the accelerometer downwards while holding it. After a beep from the MiniArm buzzer, the robotic arm will grasp the block.

Taking the "hiwonder" label on the accelerometer as the positive direction, tilt the accelerometer to the left while holding it. After a beep from the MiniArm buzzer, the robotic arm will grasp the cube and place it in the left side of the arm.

Taking the "hiwonder" label on the accelerometer as the positive direction, tilt the accelerometer to the right while holding it. After a beep from the MiniArm buzzer, the robotic arm will grasp the cube and place it in the right side of the arm.

5. Program Brief Analysis

5.1 Import Library File

```
#include <FastLED.h>
#include <Servo.h> /
#include "tone.h" //
```

```
#include <MPU6050.h>
#include "mini_servo.h"
```

Import the RGB control library, servo control library, tone library, accelerometer sensor library and robotic arm control library.

5.2 Define Pins and Create Objects

1)Create variables related to the MPU6050 accelerometer. Among them, “ax”, “ay”, and “az” represent the values of the tilt angles in the three-axis directions, while “gx”, “gy”, and “gz” represent the values of acceleration in the three-axis directions. The “radianX_last” represents the final tilt angle obtained in the X-axis direction, while “radianY_last” represents the final tilt angle obtained in the Y-axis direction.

```
MPU6050 accelgyro;
int16_t ax, ay, az;
int16_t gx, gy, gz;
float ax0, ay0, az0;
float gx0, gy0, gz0;
float ax1, ay1, az1;
float gx1, gy1, gz1;

float dx;
float dz;
int ax_offset, ay_offset, az_offset, gx_offset, gy_offset, gz_offset;
float radianX;
float radianY;
float radianZ;
float radianX_last; //final obtained X-axis inclination
float radianY_last; //final obtained Y-axis inclination
```

2)Define the tones of the two buzzers and the Arduino pins connected to the hardware. It mainly consist of six servo pins, one buzzer pin and one RGB light pin.

```
//RGB light control object
static CRGB rgbs[1];
//action group control object
HW_ACTION_CTL action_ctl;
//servo control object
Servo servos[5];

/* limit for each joint angle */
const uint8_t limt_angles[5][2] = {{0,90},{0,180},{0,180},{0,180},{0,180}};
/* the angle value actually controlled by servo */
static float servo_angles[5] = { 80,15,175,65,90 };
```

3)Create the objects of the RGB light, action group control object and servo control object. Meanwhile, variables controlled by the servo are created. For example, “limit_angle” set angle limits for each joint of robotic arm, and the “servo_angles” array is used for storing actual angle of servo with range from 0 to 180.

4）In the constructor of the action group object, the “extended_func_angles” is defined. That is the desired angle value of servo. Meanwhile, it declares relevant function used for controlling execute action group.

```
class HW_ACTION_CTL{
  public:
   /* the angle used in the secondary development routines  */
    uint8_t extended_func_angles[5] = { 80,15,175,65,90 };
    //control execute action group
    void action_set(int num);
    int action_state_get(void);
    void action_task(void);
    void read_offset(void);
    int8_t* get_offset(void);

  private:
    //action group control variables
    int action_num = 0;
    int8_t servo_offset[5];
    uint8_t eeprom_read_buf[16];
};
```

For the desired value and actual value, there may be difference in the subsequent control of servo rotation due to numerical differences. You can see the “servo_control” function called in the loop main function specifically.

If you want the servo to approach to the target position gradually, call this line of code “servo_angles[i] = servo_angles[i] * 0.85 + extended_func_angles[i] * 0.15” in the control task function.

This causes the servo to move each time by adding a portion of the servo pulse-width of the desired value to its previous position. After each movement, the current new data will be reassigned. This enables the robotic arm to stop moving when the target position is reached.

```
  for (int i = 0; i < 5; ++i) {
    if(servo_angles[i] > action_ctl.extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + action_ctl.extended_func_angles[i] * 0.1;
      if(servo_angles[i] < action_ctl.extended_func_angles[i])
        servo_angles[i] = action_ctl.extended_func_angles[i];
    }else if(servo_angles[i] < action_ctl.extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + (action_ctl.extended_func_angles[i] * 0.1 + 1);
      if(servo_angles[i] > action_ctl.extended_func_angles[i])
        servo_angles[i] = action_ctl.extended_func_angles[i];
    }

    servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
    servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
    servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
  }
}
```

5) Task functions are used for executing different control task. The “servo_control” function is used for controlling servo, the “play_tune()” function

is used for setting buzzer sound, the “tune_task()” function is used for controlling buzzer sound, “action_task()” function is used for calling action group running, and the “update_mpu6050()” is used for updating the obtained data of accelerometer.

```
static void servo_control(void); /* servo control */
void play_tune(uint16_t *p, uint32_t beat, uint16_t len); /* buzzer control interface */
void tune_task(void); /* buzzer control task */
void user_task(void); /* user task */

void update_mpu6050(void); /*update inclination sensor data*/
```

5.3 Initialization Settings

1)The “setup()” function is used for initializing the relevant hardware devices. First is the serial port, set the baud rate for communication to 115200 and the read data timeout to 500ms.

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  // set the read data timeout of serial port
  Serial.setTimeout(500);
```

2)Assign the servo IO port, use the FastLED library to initialize the RGB light on the expansion board and connect it to the RGB pin. Set the RGB color to green through “rgbs[0] = CRGB(0, 255, 0)”. Finally, use the “FastLED.show” function to display the set color.

```
// assign servo IO port
for (int i = 0; i < 5; ++i) {
  servos[i].attach(servoPins[i],500,2500);
}

//initialize the RGB light and control it
FastLED.addLeds<WS2812, rgbPin, GRB>(rgbs, 1);
rgbs[0] = CRGB(0, 0, 100);
FastLED.show();
```

3)Set the buzzer interface to output mode and call the “tone()” function to make the buzzer sound. Call “noTone()” function to stop sounding the buzzer after a delay of 100ms. Then read the deviation data of servo.

```
//initialize buzzer and beep one time
pinMode(buzzerPin, OUTPUT);
tone(buzzerPin, 1000);
delay(100);
noTone(buzzerPin);

//read deviation data of servo
action_ctl.read_offset();
```

Create an object for the accelerometer, set the ranges for angular velocity and acceleration, obtain data from each axis, and finally calibrate the data.

```
//MPU6050 configuration
Wire.begin();
accelgyro.initialize();
accelgyro.setFullScaleGyroRange(3); //set the range of angular velocity
accelgyro.setFullScaleAccelRange(1); //set the range of acceleration
delay(200);
accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);  //get current data for each axis to calibrate
ax_offset = ax;  //X-axis acceleration calibration data
ay_offset = ay;  //Y-axis acceleration calibration data
az_offset = az - 8192;  //Z-axis acceleration calibration data
gx_offset = gx; //X-axis angular velocity calibration data
gy_offset = gy; //Y-axis angular velocity calibration data
gz_offset = gz; //Z-axis angular velocity calibration data
```

5.4 User Task

After initialization, call the ultrasonic task function “touch_task” in the loop main function, which is used for adjusting action group to control the robotic arm based on the tilted angle of accelerometer. The function defines the variable “last_tick” firstly, which is mainly used for calculating the time interval.

```
// user task
void user_task(void)
{
  static uint32_t last_tick = 0;
  static uint8_t step = 0;
  static uint8_t act_num = 0;
  static uint32_t delay_count = 0;
  int color = 0;

  // time interval 100ms
  if (millis() - last_tick < 100) {
    return;
  }
  last_tick = millis();
```

Next, check if 100ms have passed since the last call to the “ultrasound_task” function. If it have passed, the following code is executed. Otherwise, the

function returns directly. The “millis” function is the time since the Arduino board started. In this way, the function ensures that the task of detecting the distance is performed every 100ms.

Then, get filtered ultrasonic measurements through the “ul.Filter” function, and store the outcome in the variable “dis”.

5.5 Execution Feedback

1)Following the steps, detect the downward tilt angle of the accelerometer in the X-axis direction firstly. If the tilt angle is greater than 40, the RGB on the control expansion board lights up red, and the buzzer beeps once to record the action group that needs to be called for action group 1. That action group is the robotic arm grasps the block. If there is no tilt or the tilt angle is too small, the robotic arm will not respond.

```
switch(step)
{
  case 0:
    if(radianX_last > 40) //grasp
    {
      rgbs[0].r = 100; //red
      rgbs[0].g = 0;
      rgbs[0].b = 0;
      FastLED.show();
      play_tune(DOC6, 300u, 1u);
      // action group 1 needs to be executed for sorting
      act_num = 1;
      step++;
    }else{ //if it is not detected
      rgbs[0].r = 0;
      rgbs[0].g = 0;
      rgbs[0].b = 0;
      FastLED.show();
    }
    break;
```

2)Enter case 1 and perform delay wait based on the value of “delay_count”. Since each time enter the case 1, the tilt angle detection task is called. And 100ms delay is performed each time when you enter the button detection task. Therefore, the case 1 will delay for 1000ms firstly. Then set the number of

action group and running it. After that, it will jump to the next stage.

```
case 1: //wait for 1s and place the wood block to the correct position.
  delay_count++;
  if(delay_count > 5)
  {
    delay_count = 0;
    // run action group
    action_ctl.action_set(act_num);
    act_num = 0;
    step++;
  }
  break;
```

3)Case 2 is the stage where it waits for the action to finish running. After completing the recognition, waiting, and calling the action group as mentioned above, it checks if the “action_num” variable is 0 to determine if the action group has finished running. After the action group finishes, enter the next stage.

```
case 2: //wait for resetting the action state
  if(action_ctl.action_state_get() == 0)
  {
    step++;
  }
  break;
```

4)Case 3 is the stage detects tilt angle of the accelerometer in the Y-axis direction, which also means the left and right direction.

```
case 3:
  if(radianY_last > 50) //right
  {
    rgbs[0].r = 0;
    rgbs[0].g = 100;
    rgbs[0].b = 0;
    FastLED.show();
    play_tune(DOC6, 300u, 1u);
    // action group 2 needs to be executed for sorting
    action_ctl.action_set(2);
    step++;
  }else if(radianY_last < -50) //left
  {
    rgbs[0].r = 0;
    rgbs[0].g = 0;
    rgbs[0].b = 100;//blue
    FastLED.show();
    play_tune(DOC6, 300u, 1u);
    // action group 3 needs to be executed for sorting
    action_ctl.action_set(3);
    step++;
  }
  break;
```

When the tilt angle of the accelerometer Y-axis is greater than 50, the acceleration sensor tilts to the left and RGB light up green. After emitting a beep from the buzzer, record the required action group as action group 2, which means placing block to the left.

When the tilt angle of the accelerometer Y-axis is less than -50, the acceleration sensor tilts to the right and RGB light up blue. After emitting a beep from the buzzer, record the required action group as action group 3, which means placing block to the right.

5)Case 4 is the stage waiting for the action status to be cleared. When the action group finishes running, the RGB light on the expansion board will be cleared. And the variable of the current stage of “step” will be set to 0, and the execution will start from stage 0 again.

```
    case 4: //wait for resetting the action state
      if(action_ctl.action_state_get() == 0)
      {
        FastLED.clear();
        step = 0;
      }
      break;
    default:
      step = 0;
      break;
  }
}
```

5.6 Servo Control Task

1)In the servo control task, the same variable “last_tick” is used for delay processing.

```
void servo_control(void) {
  static uint32_t last_tick = 0;
  if (millis() - last_tick < 20) {
    return;
  }
  last_tick = millis();
```

2)Next, get the desired value of the servo through “action_ctl” object. Note that the desired value at this point is the result of the desired value plus the deviation outcome of servo. So this desired value is the actual desired value. Finally, the actual desired value of the servo is assigned to the actual control

angle. Then pass it to the servo object to actually control the rotation of the servo.

```
  for (int i = 0; i < 5; ++i) {
    if(servo_angles[i] > action_ctl.extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + action_ctl.extended_func_angles[i] * 0.1;
      if(servo_angles[i] < action_ctl.extended_func_angles[i])
        servo_angles[i] = action_ctl.extended_func_angles[i];
    }else if(servo_angles[i] < action_ctl.extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + (action_ctl.extended_func_angles[i] * 0.1 + 1);
      if(servo_angles[i] > action_ctl.extended_func_angles[i])
        servo_angles[i] = action_ctl.extended_func_angles[i];
    }
    servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
    servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
    servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
  }
}
```

Let's further explain the ideal desired value and actual desired value. In the "action_ctl" object, "extended_func_angles" is the ideal desired value, which excludes deviation due to servo structure. In the actual project,

servos often have a certain range of deviations due to their mechanical structure. To achieve more accurate control of the servo angle, we calculate the additional angle needed for the servo to reach the ideal position by adjusting for this deviation. This additional angle can be positive, negative, or zero. Then, during actual servo control, add this deviation value to the ideal expected angle, resulting in the actual expected angle.

6. Function Extension

We can modify the condition to: if (radianX_last < -40), so that when the accelerometer sensor tilts towards the negative direction of the X-axis, the grabbing action will be executed.

```
switch(step)
{
  case 0:
    if(radianX_last > 40) //grasp
    {
      rgbs[0].r = 100; //red
      rgbs[0].g = 0;
      rgbs[0].b = 0;
      FastLED.show();
      play_tune(DOC6, 300u, 1u);
      // action group 1 needs to be executed for sorting
      act_num = 1;
      step++;
    }else{ //if it is not detected
      rgbs[0].r = 0;
      rgbs[0].g = 0;
      rgbs[0].b = 0;
      FastLED.show();
    }
    break;
```

7. FAQ

Q: since the code was upload, the gyroscope accelerometer sensor is not responding to rotation.

A: please check that you have connected the 4-pin cable to the I2C interface and turn the knob on the expansion board to the initial position.

Q: the gyroscope accelerometer sensor is tilted but not responding.

A: please ensure that the sensor is in a horizontal position before use.